28-1-2026

Tema 3

Instalación y Configuración Hibérnate

Alberto Fernández Martínez

Índice

Instrucciones tema 3 (Hibérnate).....2

Paso1: Creamos el proyecto en Intellij Ultimate, en nuestro caso va a ser un proyecto Maven Archetype, una vez se abra la pantalla de configuración ponemos el nombre, la localización donde queremos que se guarde el proyecto elegimos el jdk (que es la versión de nuestro java en mi caso es java 21) catalogo internal, en Archetype es cogemos la versión quickstart y le damos a crear. 2

Paso2 Agregamos la base de datos fct al MySQL 3

Paso3 Ahora creamos la estructura del Intellij en src-java-org. Tema3(dentro tienes 2 carpetas configuración y model y un archivo Main.java) dentro de la carpeta main tenemos la carpeta resources si no está hay que crearla dentro del main y dentro de la carpeta resources tenemos el archivo (hibernate.cfg.xml) 4

Paso 4 Añadimos las dependencias al pom de hibérnate y MySQL......6

Paso5 En la carpeta de Configuración creamos el archivo HibernateUtil donde vamos a crear nuestra conexión con la base de datos MySQL con nuestro usuario, contraseña, puerto y nombre de la base de datos......7

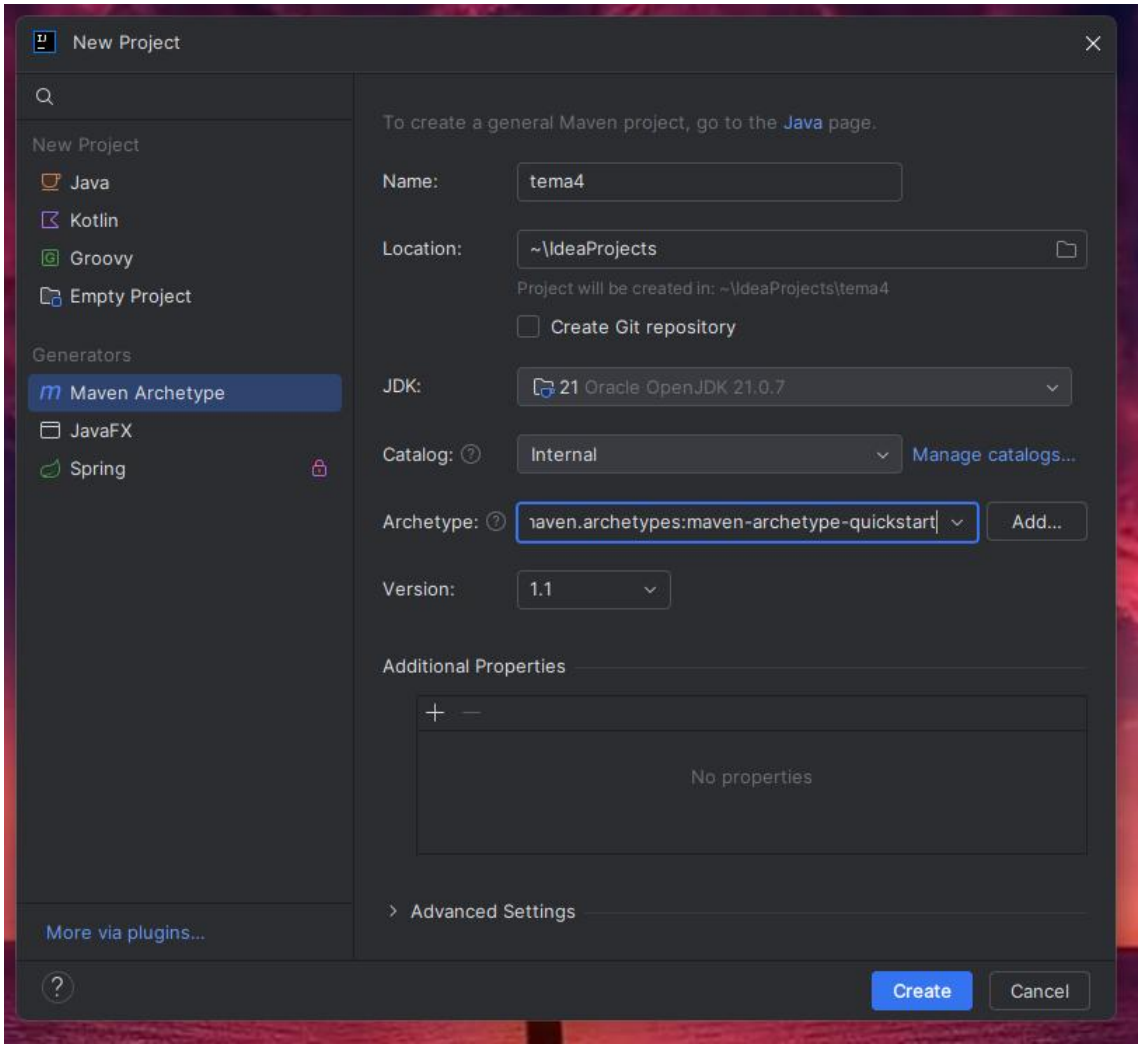
Paso6 Ahora creamos la carpeta model que tendrá el archivo java Alumno que es el que coincide con la tabla de la base de datos alumno......8

Paso 7 dentro de la carpeta org. Tema3 creamos el archivo main que será el archivo app que tendremos que cambiar el nombre por el de main es la clase ejecutadora. 14

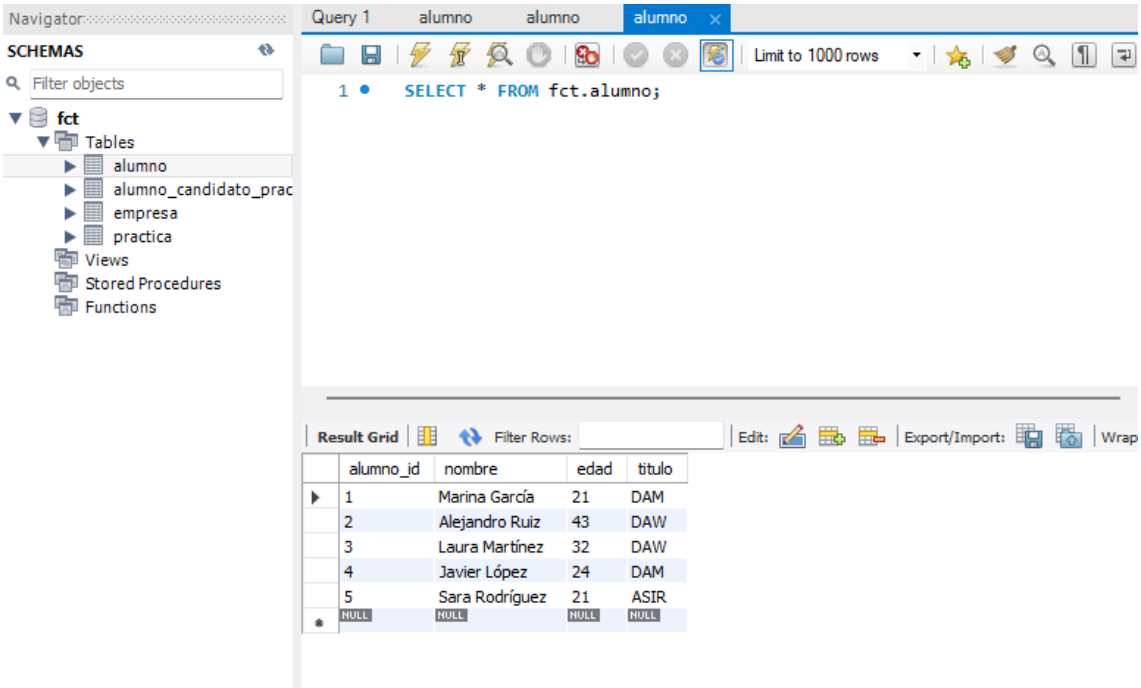
Biografía 16

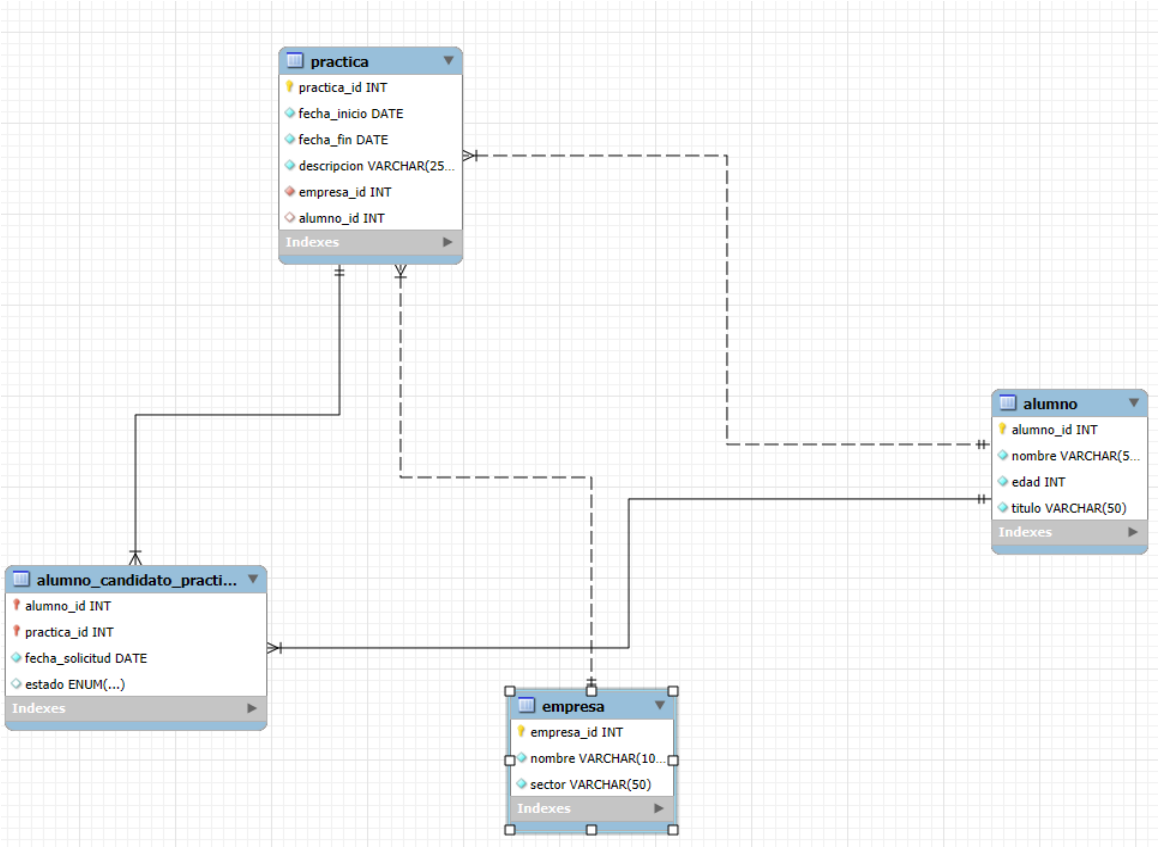
Instrucciones tema 3 (Hibérnate)

Paso1: Creamos el proyecto en Intellij Ultimate, en nuestro caso va a ser un proyecto Maven Archetype, una vez se abra la pantalla de configuración ponemos el nombre, la localización donde queremos que se guarde el proyecto elegimos el jdk (que es la versión de nuestro java en mi caso es java 21) catalogo internal, en Archetype es cogemos la versión quickstart y le damos a crear.



Paso2 Agregamos la base de datos fct al MySQL





Paso3 Ahora creamos la estructura del Intellij en src-java-org. Tema3(dentro tienes 2 carpetas configuración y model y un archivo Main.java) dentro de la carpeta main tenemos la carpeta resources si no está hay que crearla dentro del main y dentro de la carpeta resources tenemos el archivo (hibernate.cfg.xml).

```xml
<?xml version="1.0" encoding="utf-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
        "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
        "https://hibernate.org/dtd/hibernate-configuration-3.0.dtd">
<hibernate-configuration>
    <session-factory>

        <!-- JDBC -->
        <property name="hibernate.connection.driver_class">com.mysql.cj.jdbc.Driver</property>
        <property name="hibernate.connection.url">
            jdbc:mysql://localhost:3307/fct?useSSL=false&amp;allowPublicKeyRetrieval=true&amp;serverTimezone=UTC
        </property>
        <property name="hibernate.connection.username">app</property>
        <property name="hibernate.connection.password">123</property>


        <!-- NO toques el esquema, solo valida que coincide con la BD existente -->
        <property name="hibernate.hbm2ddl.auto">validate</property>

        <!-- Logging SQL -->
        <property name="hibernate.show_sql">true</property>
        <property name="hibernate.format_sql">true</property>
        <property name="hibernate.highlight_sql">true</property>

        <!-- Mapeo de entidades -->
        <mapping class="org.tema4.model.Alumno"/>
        <mapping class="org.tema4.model.Empresa"/>
        <mapping class="org.tema4.model.Practica"/>
        <mapping class="org.tema4.model.AlumnoCandidatoPractica"/>

    </session-factory>
</hibernate-configuration>
```

También tenemos el archivo (logging. proerties) que es para quitar los avisos por terminal no es necesario usarlo. Para usarlo te va a run en la parte **Run → Edit Configurations… → tu Main → VM options** y luego en la ventana que te sale.

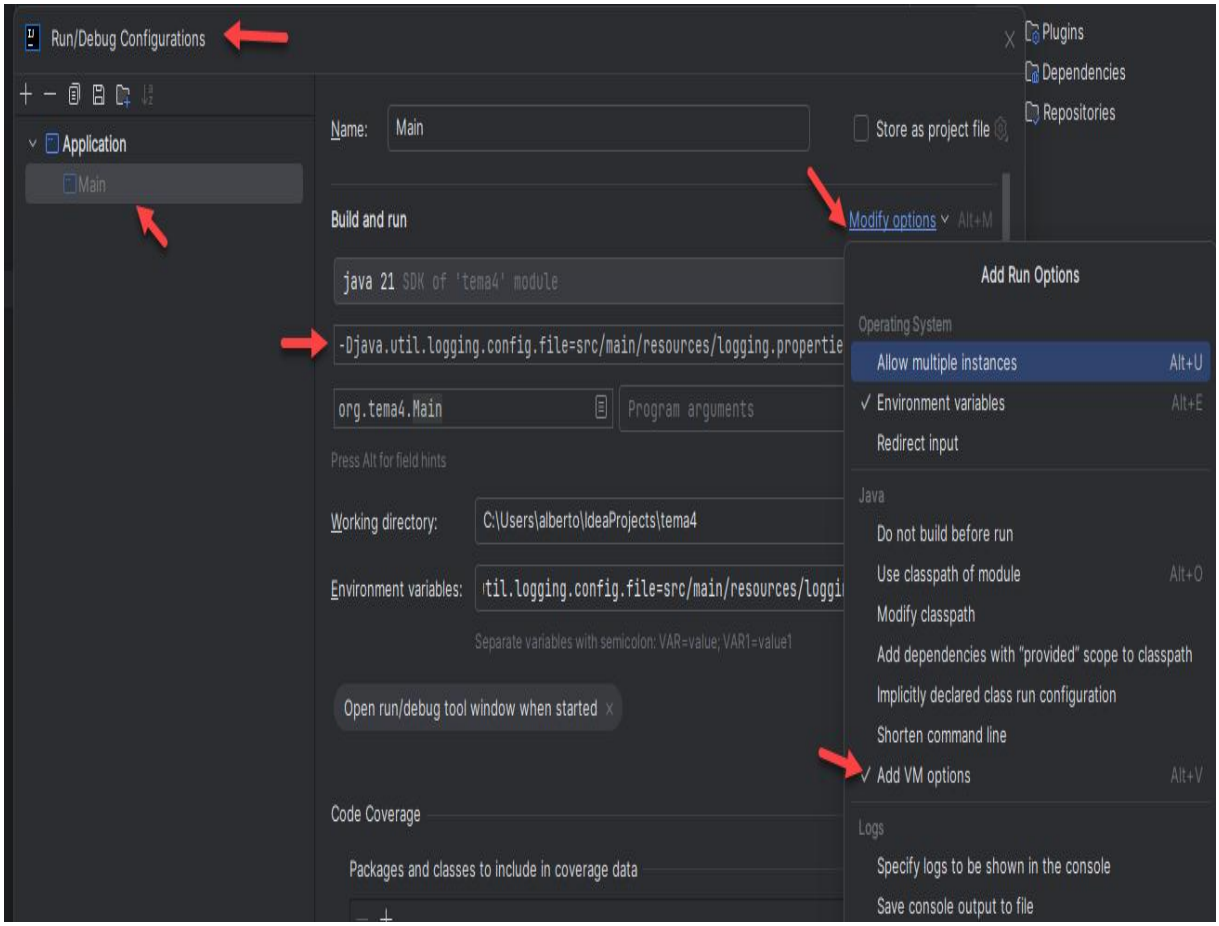
En esa ventana, arriba a la derecha donde pone **"Modify options"** (lo tienes visible).

Click en **Modify options**.

En el desplegable, marca **"Add VM options"** (o "VM options", depende del idioma).

Te aparecerá un campo nuevo llamado **VM options**.

Ahí pega esto: -Djava.util. logging.config. file=src/main/resources/logging.properties


Run/Debug Configurations
Application
Main
Name:
Main
Store as project file
Build and run
Modify options
Alt+M
java 21 SDK of 'tema4' module
-Djava.util.logging.config.file=src/main/resources/logging.propertie
org.tema4.Main
Program arguments
Press Alt for field hints
Working directory:
C:\Users\alberto\IdeaProjects\tema4
Environment variables:
til.logging.config.file=src/main/resources/loggi
Separate variables with semicolon: VAR=value; VAR1=value1
Open run/debug tool window when started
Code Coverage
Packages and classes to include in coverage data
Plugins
Dependencies
Repositories
Add Run Options
Operating System
Allow multiple instances
Alt+U
Environment variables
Alt+E
Redirect input
Java
Do not build before run
Use classpath of module
Alt+O
Modify classpath
Add dependencies with "provided" scope to classpath
Implicitly declared class run configuration
Shorten command line
Add VM options
Alt+V
Logs
Specify logs to be shown in the console
Save console output to file

Paso 4 Añadimos las dependencias al pom de hibérnate y MySQL.

```xml
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>es.tuempresa</groupId>
    <artifactId>tema4</artifactId>
    <version>1.0-SNAPSHOT</version>
    <packaging>jar</packaging>

    <name>tema4</name>

    <properties>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <maven.compiler.release>21</maven.compiler.release>

        <!-- Versiones -->
        <hibernate.version>7.2.1.Final</hibernate.version>
        <mysql.connector.version>9.5.0</mysql.connector.version>
        <jakarta.persistence.version>3.2.0</jakarta.persistence.version>
        <slf4j.version>2.0.17</slf4j.version>
        <junit.jupiter.version>5.11.4</junit.jupiter.version>
    </properties>

    <dependencies>
        <!-- Hibernate ORM -->
        <dependency>
            <groupId>org.hibernate.orm</groupId>
            <artifactId>hibernate-core</artifactId>
            <version>${hibernate.version}</version>
        </dependency>

        <!-- API JPA -->
        <dependency>
            <groupId>jakarta.persistence</groupId>
            <artifactId>jakarta.persistence-api</artifactId>
            <version>${jakarta.persistence.version}</version>
        </dependency>

        <!-- Driver MySQL -->
        <dependency>
            <groupId>com.mysql</groupId>
            <artifactId>mysql-connector-j</artifactId>
            <version>${mysql.connector.version}</version>
        </dependency>

        <!-- Logging sencillo a consola -->
        <dependency>
            <groupId>org.slf4j</groupId>
            <artifactId>slf4j-simple</artifactId>
            <version>${slf4j.version}</version>
        </dependency>

        <!-- JUnit 5 -->
        <dependency>
            <groupId>org.junit.jupiter</groupId>
            <artifactId>junit-jupiter</artifactId>
            <version>${junit.jupiter.version}</version>
            <scope>test</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <!-- Necesario para ejecutar tests JUnit 5 -->
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-surefire-plugin</artifactId>
                <version>3.5.2</version>
            </plugin>
        </plugins>
    </build>

</project>
```

Paso5 En la carpeta de Configuración creamos el archivo HibernateUtil donde vamos a crear nuestra conexión con la base de datos MySQL con nuestro usuario, contraseña, puerto y nombre de la base de datos.

```java
package org.tema4.configuracion;

import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;

public final class HibernateUtil {

    private static final String HIBERNATE_CFG_XML = "hibernate.cfg.xml";

    private static final SessionFactory SESSION_FACTORY = buildSessionFactory();

    private HibernateUtil() {
        // utility
    }

    private static SessionFactory buildSessionFactory() {
        try {
            // Carga explícita desde src/main/resources/hibernate.cfg.xml
            return new Configuration()
                    .configure(HIBERNATE_CFG_XML)
                    .buildSessionFactory();
        } catch (Throwable ex) {
            // Importante: muestra stacktrace completo (causa real)
            System.err.println("Error inicializando SessionFactory con " + HIBERNATE_CFG_XML);
            ex.printStackTrace(System.err);
            throw new ExceptionInInitializerError(ex);
        }
    }

    public static SessionFactory getSessionFactory() {
        return SESSION_FACTORY;
    }

    public static Session openSession() {
        return SESSION_FACTORY.openSession();
    }

    public static void shutdown() {
        if (SESSION_FACTORY != null && !SESSION_FACTORY.isClosed()) {
            SESSION_FACTORY.close();
        }
    }
}
```

Paso6 Ahora creamos la carpeta model que tendrá el archivo java Alumno que es el que coincide con la tabla de la base de datos alumno.

```java
package org.tema4.model;

import jakarta.persistence.*;

@Entity
@Table(name = "Alumno")
public class Alumno {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "alumno_id")
    private Integer id; // INT en BD -> Integer en Java

    @Column(name = "nombre", nullable = false, length = 50)
    private String nombre;

    @Column(name = "edad", nullable = false)
    private int edad;

    @Column(name = "titulo", nullable = false, length = 50)
    private String titulo;

    protected Alumno() {
        // requerido por Hibernate
    }

    public Alumno(String nombre, int edad, String titulo) {
        this.nombre = nombre;
        this.edad = edad;
        this.titulo = titulo;
    }

    public Integer getId() { return id; }
    public String getNombre() { return nombre; }
    public int getEdad() { return edad; }
    public String getTitulo() { return titulo; }

    @Override
    public String toString() {
        return "Alumno{id=" + id + ", nombre='" + nombre + "', edad=" + edad + ", titulo='" + titulo + "'}";
    }
}
```

Ahora creamos la clase AlumnoCandidatoPractica.

```java
package org.tema4.model;

import jakarta.persistence.*;

import java.time.LocalDate;
import java.util.Objects;

@Entity
@Table(name = "Alumno_Candidato_Practica")
public class AlumnoCandidatoPractica {

    @EmbeddedId
    private AlumnoCandidatoPracticaId id = new AlumnoCandidatoPracticaId();

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @MapsId("alumnoId")
    @JoinColumn(name = "alumno_id", nullable = false)
    private Alumno alumno;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @MapsId("practicaId")
    @JoinColumn(name = "practica_id", nullable = false)
    private Practica practica;

    @Column(name = "fecha_solicitud", nullable = false)
    private LocalDate fechaSolicitud;

    @Enumerated(EnumType.STRING)
    @Column(name = "estado", nullable = false)
    private EstadoCandidatura estado;

    protected AlumnoCandidatoPractica() { }

    public AlumnoCandidatoPractica(Alumno alumno, Practica practica, LocalDate fechaSolicitud, EstadoCandidatura estado) {
        this.alumno = Objects.requireNonNull(alumno, "alumno no puede ser null");
        this.practica = Objects.requireNonNull(practica, "practica no puede ser null");
        this.fechaSolicitud = Objects.requireNonNull(fechaSolicitud, "fechaSolicitud no puede ser null");
        this.estado = Objects.requireNonNull(estado, "estado no puede ser null");
        // NO construyas id con alumno.getId()/practica.getId(): @MapsId lo sincroniza
    }

    public AlumnoCandidatoPracticaId getId() { return id; }
    public Alumno getAlumno() { return alumno; }
    public Practica getPractica() { return practica; }
    public LocalDate getFechaSolicitud() { return fechaSolicitud; }
    public EstadoCandidatura getEstado() { return estado; }

    public void setEstado(EstadoCandidatura estado) {
        this.estado = Objects.requireNonNull(estado, "estado no puede ser null");
    }
}
```

Ahora creamos la clase AlumnoCandidatoPracticaId.

```java
package org.tema4.model;

import jakarta.persistence.Column;
import jakarta.persistence.Embeddable;

import java.io.Serializable;
import java.util.Objects;

@Embeddable
public class AlumnoCandidatoPracticaId implements Serializable {

    @Column(name = "alumno_id")
    private Integer alumnoId;   // <-- antes Long

    @Column(name = "practica_id")
    private Integer practicaId; // <-- antes Long

    protected AlumnoCandidatoPracticaId() {}

    public AlumnoCandidatoPracticaId(Integer alumnoId, Integer practicaId) {
        this.alumnoId = alumnoId;
        this.practicaId = practicaId;
    }

    public Integer getAlumnoId() { return alumnoId; }
    public Integer getPracticaId() { return practicaId; }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof AlumnoCandidatoPracticaId that)) return false;
        return Objects.equals(alumnoId, that.alumnoId)
                && Objects.equals(practicaId, that.practicaId);
    }

    @Override
    public int hashCode() {
        return Objects.hash(alumnoId, practicaId);
    }
}
```

Ahora Creamos la clase Empresa

```java
package org.tema4.model;

import jakarta.persistence.*;

@Entity
@Table(name = "Empresa")
public class Empresa {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "empresa_id")
    private Integer id; // INT -> Integer

    @Column(name = "nombre", nullable = false, length = 100)
    private String nombre;

    @Column(name = "sector", nullable = false, length = 50)
    private String sector;

    protected Empresa() { }

    public Empresa(String nombre, String sector) {
        this.nombre = nombre;
        this.sector = sector;
    }

    public Integer getId() { return id; }
    public String getNombre() { return nombre; }
    public String getSector() { return sector; }

    @Override
    public String toString() {
        return "Empresa{id=" + id + ", nombre='" + nombre + "', sector='" + sector + "'}";
    }
}

```

Ahora creamos la clase EstadoCandidatura

```
1  package org.tema4.model;
2
3  public enum EstadoCandidatura {
4      Pendiente,
5      Aceptado,
6      Rechazado
7  }
```

Ahora creamos la clase Practica

```java
package org.tema4.model;

import jakarta.persistence.*;

import java.time.LocalDate;

@Entity
@Table(name = "Practica")
public class Practica {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "practica_id")
    private Integer id; // INT en BD -> Integer

    @Column(name = "fecha_inicio", nullable = false)
    private LocalDate fechaInicio;

    @Column(name = "fecha_fin", nullable = false)
    private LocalDate fechaFin;

    @Column(name = "descripcion", nullable = false, length = 255)
    private String descripcion;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "empresa_id", nullable = false)
    private Empresa empresa;

    /**
     * En tu SQL: alumno_id puede ser NULL y además es UNIQUE,
     * así que lo modelamos como OneToOne opcional.
     */
    @OneToOne(fetch = FetchType.LAZY, optional = true)
    @JoinColumn(name = "alumno_id", unique = true, nullable = true)
    private Alumno alumno;

    protected Practica() { }

    public Practica(LocalDate fechaInicio, LocalDate fechaFin, String descripcion, Empresa empresa) {
        this.fechaInicio = fechaInicio;
        this.fechaFin = fechaFin;
        this.descripcion = descripcion;
        this.empresa = empresa;
    }

    public Integer getId() { return id; }
    public LocalDate getFechaInicio() { return fechaInicio; }
    public LocalDate getFechaFin() { return fechaFin; }
    public String getDescripcion() { return descripcion; }
    public Empresa getEmpresa() { return empresa; }
    public Alumno getAlumno() { return alumno; }

    public void asignarAlumno(Alumno alumno) {
        this.alumno = alumno;
    }

    @Override
    public String toString() {
        return "Practica{id=" + id + ", fechaInicio=" + fechaInicio + ", fechaFin=" + fechaFin +
                ", descripcion='" + descripcion + "'}";
    }
}
```

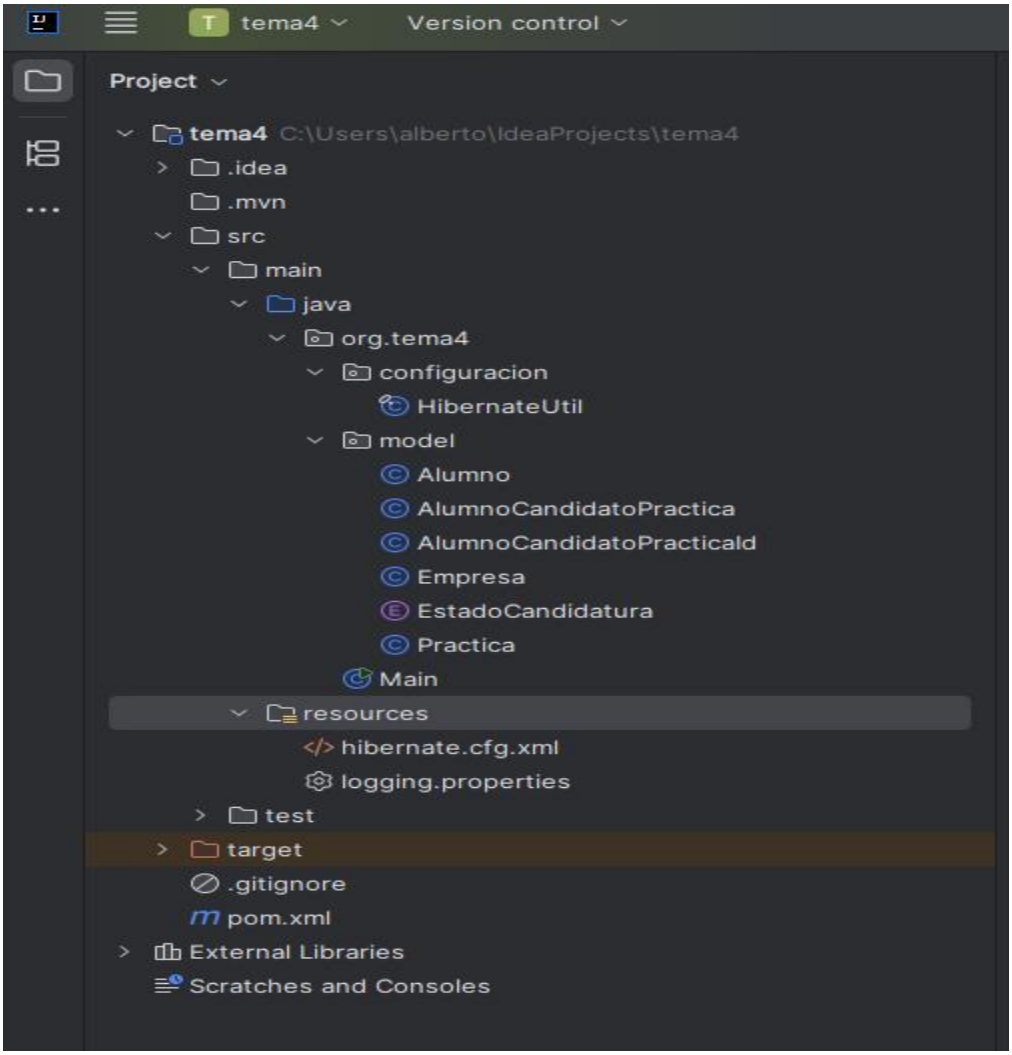
Paso 7 dentro de la carpeta org. Tema3 creamos el archivo main que será el archivo app que tendremos que cambiar el nombre por el de main es la clase ejecutadora.

```java
package org.tema4;

import org.hibernate.Session;
import org.hibernate.Transaction;
import org.tema4.configuracion.HibernateUtil;
import org.tema4.model.Alumno;

import java.util.List;
import java.util.Scanner;

public class Main {

    public static void main(String[] args) {
        try (Scanner sc = new Scanner(System.in)) {
            boolean salir = false;

            while (!salir) {
                System.out.println("""
                        \n--- MENÚ ---
                        1) Listar alumnos
                        2) Insertar alumno
                        3) Borrar alumno por id
                        0) Salir
                        """);

                System.out.print("Opción: ");
                String op = sc.nextLine().trim();

                switch (op) {
                    case "1" -> listar();
                    case "2" -> insertar(sc);
                    case "3" -> borrar(sc);
                    case "0" -> salir = true;
                    default -> System.out.println("Opción no válida.");
                }
            }
        } finally {
            HibernateUtil.shutdown();
        }
    }

    private static void listar() {
        try (Session session = HibernateUtil.openSession()) {
            List<Alumno> alumnos = session.createQuery("from Alumno order by id", Alumno.class).getResultList();
            alumnos.forEach(System.out::println);
        }
    }

    private static void insertar(Scanner sc) {
        System.out.print("Nombre: ");
        String nombre = sc.nextLine().trim();

        System.out.print("Edad: ");
        int edad = Integer.parseInt(sc.nextLine().trim());

        System.out.print("Título: ");
        String titulo = sc.nextLine().trim();

        Transaction tx = null;
        try (Session session = HibernateUtil.openSession()) {
            tx = session.beginTransaction();

            Alumno a = new Alumno(nombre, edad, titulo);
            session.persist(a);

            tx.commit();
            System.out.println("Insertado con id=" + a.getId());
        } catch (Exception e) {
            if (tx != null) tx.rollback();
            throw e;
        }
    }

    private static void borrar(Scanner sc) {
        System.out.print("Id a borrar: ");
        int id = Integer.parseInt(sc.nextLine().trim());

        Transaction tx = null;
        try (Session session = HibernateUtil.openSession()) {
            tx = session.beginTransaction();

            Alumno a = session.find(Alumno.class, id);
            if (a == null) {
                System.out.println("No existe alumno con id=" + id);
            } else {
                session.remove(a);
                System.out.println("Borrado alumno id=" + id);
            }

            tx.commit();
        } catch (Exception e) {
            if (tx != null) tx.rollback();
            throw e;
        }
    }
}
```

Como nota: dentro del archivo hibernate.cfg.xml en la parte del mapeo de entidades son las tablas que hay en la base de datos si hay más tablas hay que meterlas todas.

```
<!-- Mapeo de entidades -->
<mapping class="org.tema4.model.Alumno"/>
<mapping class="org.tema4.model.Empresa"/>
<mapping class="org.tema4.model.Practica"/>
<mapping class="org.tema4.model.AlumnoCandidatoPractica"/>
```

Y así se nos quedaría la estructura de nuestro proyecto en el Intellij



Biografía

He sacado la información de apuntes de la clase y por internet.

En mi caso he optado por instalar y configurar el hibérnate mediante dependencia en el pom del proyecto.